

IEMS 490 - Foundations of Large Language Models

Final Project: Building an LLM-Powered Coding Agent

1. Overview

In this project, you will design and implement an LLM-powered coding agent: a system that takes natural-language instructions from a user and carries out software engineering tasks such as writing code, editing files, and running commands.

Modern coding agents are more than a chatbot wrapper around a language model. They are structured systems that combine an LLM backbone with a harness layer that defines what the model can see and do. Your goal is to build this harness and demonstrate that your agent can complete coding tasks end-to-end.

2. Motivation

Large language models can generate code, but a bare model has no way to read files, execute programs, or interact with the outside world. A coding agent bridges this gap by equipping the LLM with a set of capabilities (tools) and a control loop that orchestrates their use.

The central engineering challenge is not the model itself, but the harness: what tools the agent has access to, how it selects among them, and how higher-level workflows are composed from primitive actions. This project asks you to build that harness.

IEMS 490 - Foundations of Large Language Models

Final Project: Building an LLM-Powered Coding Agent

3. System Architecture

Your agent should include the following components:

3.1 Tool System

Tools are the atomic capabilities your agent can invoke. Each tool has a name, a structured input schema (e.g., JSON), and an execution function. The LLM decides which tool to call and with what arguments; the harness executes the tool and returns the result.

You must implement at least the following core tools:

- File Read: read the contents of a file given its path.
- File Write / Edit: create or modify files in the workspace.
- Shell Execute: run a shell command and capture stdout/stderr.
- Search: find files by name pattern or search file contents by keyword.

Each tool should have a clear schema so the LLM knows how to call it. You are encouraged to add additional tools (e.g., web search, git operations).

3.2 Skill System (Composable Workflows)

Skills are higher-level, reusable workflows that combine multiple tool calls into a coherent routine. While a tool is a single action, a skill is a multi-step recipe that the user can invoke by name.

Examples of skills:

- /commit: stage changed files, generate a commit message from the diff, and create a git commit.
- /test: detect the project's test framework, run the test suite, and summarize failures.
- /review: read a code diff, analyze it for bugs and style issues, and produce a structured review.

Skills should be defined in a way that makes it easy to add new ones without modifying the core agent code (e.g., as prompt templates or configuration files). You must implement at least 2 skills.

3.3 Agent Loop

The agent loop is the main control flow. At a high level, the loop repeats:

1. Observe: gather context (user message, tool results, file contents).
2. Think: send context to the LLM, which decides the next action.
3. Act: execute the chosen tool or skill.
4. Check: determine if the task is complete; if not, loop.

Your agent should support multi-step reasoning: most real tasks require more than one tool call. The agent should be able to chain multiple actions based on intermediate results.

3.4 Safety and Permissions

A coding agent that can execute shell commands and modify files is powerful but risky. Your system should include a basic permission model:

- Distinguish between safe actions (e.g., reading a file) and potentially dangerous ones (e.g., deleting files, running arbitrary commands).
- Require user confirmation before executing dangerous actions.

The permission model does not need to be elaborate, but it should be present.

IEMS 490 - Foundations of Large Language Models

Final Project: Building an LLM-Powered Coding Agent

4. Deliverables

Your submission should include:

1. Code Repository: a well-organized repository containing your agent implementation. Include a README with setup and usage instructions.
2. Report (3-5 pages) covering:
 - a. System architecture diagram showing how the components fit together.
 - b. Tool and skill design: what you implemented and how they work.
 - c. Examples of your agent completing tasks, including at least one failure case and what caused it.
 - d. Reflection: what worked well, what was harder than expected, and what you would improve.
3. Demo Video (5-10 minutes): a screen recording of your agent completing at least two tasks end-to-end with brief narration.

5. Suggested Tasks for Demo

You are free to choose your own tasks. Here are some suggestions at varying difficulty:

- Read a Python file and add type annotations to all functions.
- Find all TODO comments in a repository and list them.
- Given a failing test, read the test and source code, diagnose the bug, and produce a fix.
- Implement a small feature described in plain English, including writing tests.
- Set up a new project from a natural-language specification (create files, write code, run tests).

6. Grading Rubric

The project will be graded on the following criteria (100 points total):

- Tool System (25 pts): Are the core tools implemented correctly? Do they have clear schemas? Do they handle basic errors (e.g., file not found)?
- Skill System (20 pts): Are there at least 2 working skills? Are they defined in an extensible way (not hard-coded in the agent loop)?
- Agent Loop (20 pts): Can the agent chain multiple tool calls to complete a task? Does the loop terminate correctly?
- Safety (5 pts): Does the agent ask for confirmation before dangerous actions?
- Report (15 pts): clarity, architecture diagram, examples, and honest reflection on limitations.
- Demo (15 pts): clear demonstration of the agent working on real tasks with narration.

7. Logistics

- Team size: 1-3 students.
- You may use any LLM API (OpenAI, Anthropic, open-source models, etc.) as the backbone. The focus of this project is the harness, not the model.
- You may use existing libraries for HTTP requests, JSON parsing, git, etc., but the core agent loop, tool definitions, and skill system must be your own implementation.
- Submission: upload your code repository (zip or GitHub link), report (PDF), and demo video link to Canvas by the deadline.
- Late policy: 10% penalty per day, up to 3 days. No submissions accepted after 3 days.

8. Tips

- Start simple. Get a basic loop working with File Read and Shell Execute before adding more tools.
- Test each tool independently before plugging it into the agent loop.
- Use structured output (e.g., JSON tool calls) to make parsing reliable.
- Log every LLM call and tool invocation -- this makes debugging much easier.
- Keep your skills small and composable. A skill that tries to do too much is hard to debug.